Inventory Management v4

Created	@December 7, 2025 6:40 PM
Tags	project
Created by	Ⓚ K K
Status	Not started

RESOURCES / INVENTORY MANAGEMENT

THEORY

1. What the Project *Actually Is*

Project-A is an Inventory / Resource Management System.

At its core, it answers four business questions:

1. **What items/resources do I have?**
2. **How much of each do I currently have?**
3. **What changed my inventory, when, and why?**
4. **Who is allowed to see or modify this data?**

This is the same category of software used by:

- warehouses
- hospitals
- labs
- small businesses
- libraries
- internal company tools

It's operational software.

---

2. What Problem It *Solves*

Without such a system:

- Stock updates are manual
- Errors go unnoticed
- No history of changes
- No accountability
- No alerts for low stock

System introduces:

- **structure**
- **traceability**
- **rules**
- **control**

---

3. Core *Concepts* Behind the System

There are **three fundamental ideas** behind how this system works.

Concept 1: Items Are Static, Transactions Are Dynamic

- **Items** represent *things that exist*
- **Transactions** represent *events that change those things*

This separation is extremely important.

Does not directly edit stock randomly.

Stock changes **only through transactions**.

Systems maintain **integrity**.

Concept 2: Inventory Is a Derived State

The quantity of item;

It is derived from:

- initial stock
- plus all **IN transactions**
- minus all **OUT transactions**

Concept 3: Rules Live in the Backend, Not the UI

Examples:

- Cannot remove more stock than exists
- Only authorized users can create transactions
- Certain users can only view data

These are **business rules**, and they belong in:

- models
- services
- backend logic

Not in HTML or forms.

---

4. How the System *Works*

Step 1: Item Creation

An admin creates an item:

- Name: “Printer Ink”
- Initial quantity: 50

This creates:

- One row in the **Items** table

Step 2: Transaction Occurs

Someone uses 3 units.

Instead of editing `quantity = 47` directly:

- A **Transaction** is created:
  - item = Printer Ink
  - type = OUT
  - quantity = 3
  - timestamp = now

Backend logic:

- Validates quantity
- Updates item stock accordingly

Step 3: Audit Trail Exists Automatically

Later:

- Who used the ink?
- When?
- How often?
- Was stock ever negative?

*Because **transactions preserve history**.*

---

5. Database *Design*

Items Table

Represents *what exists*.

- Stable
- Low frequency of change
- Central reference point

Transactions Table

Represents *what happens*.

- High frequency
- Time-based
- Always linked to an item
- Never edited casually

The **foreign key** enforces:

> *"A transaction cannot exist without a valid item."*

That's **relational integrity.**

---

6. Authentication & Roles

Not everyone should:

- delete items
- adjust stock
- view analytics

So:

- **Admin**: full control
- **Staff**: limited actions

---

7. Analytics & Low-Stock *View*

It's:

- filtering
- aggregation
- conditional logic

Examples:

- Items where quantity < threshold
- Recent transactions
- Fast-moving items

This e**xtract meaning from data**, not just store it.

---

8. Deployment

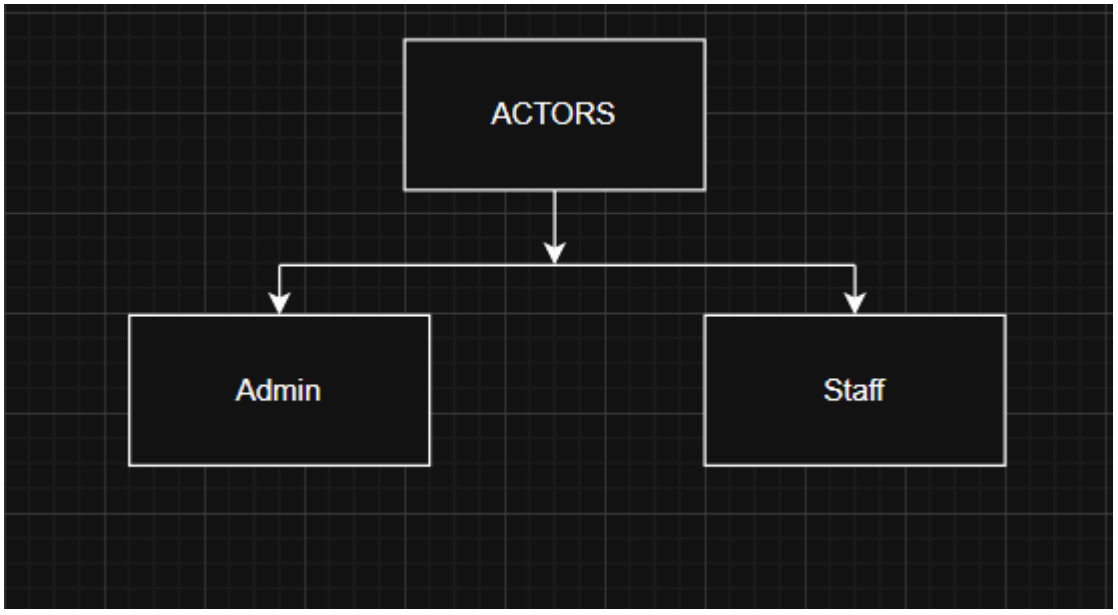
Deployment proves:

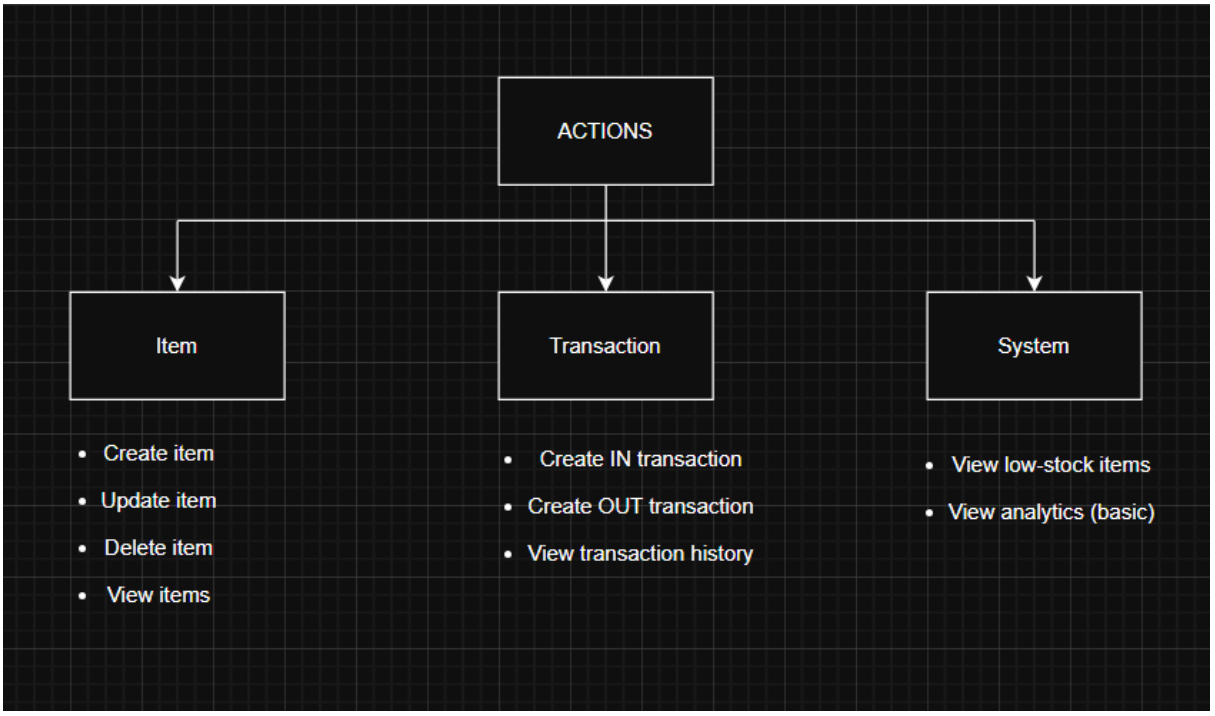
- Understanding environment separation
- Configuring databases
- Managing secrets
- Code runs outside your laptop

*A deployed system is no longer a "project". It's **software**.*

Diagrams

1. Access Control

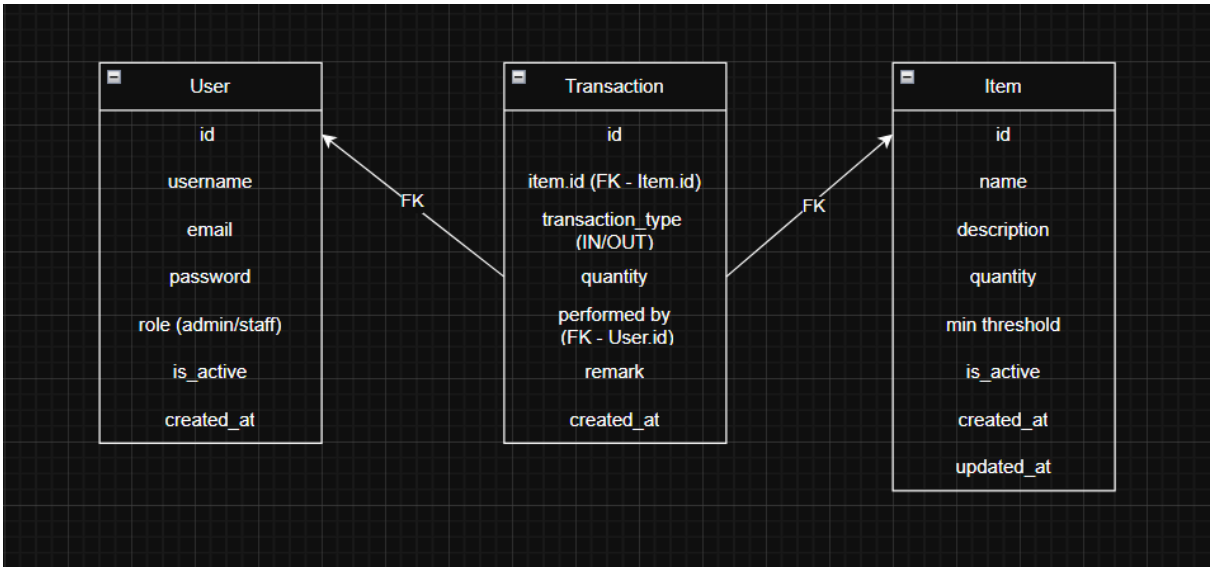




2. Role & Permission Matrix

Action	Admin	Staff
Create item	✓	✗
Update item	✓	✗
Delete item	✓	✗
View items	✓	✓
IN transaction	✓	✓
OUT transaction	✓	✓
View transactions	✓	✓
View low-stock	✓	✓
View analytics	✓	✗

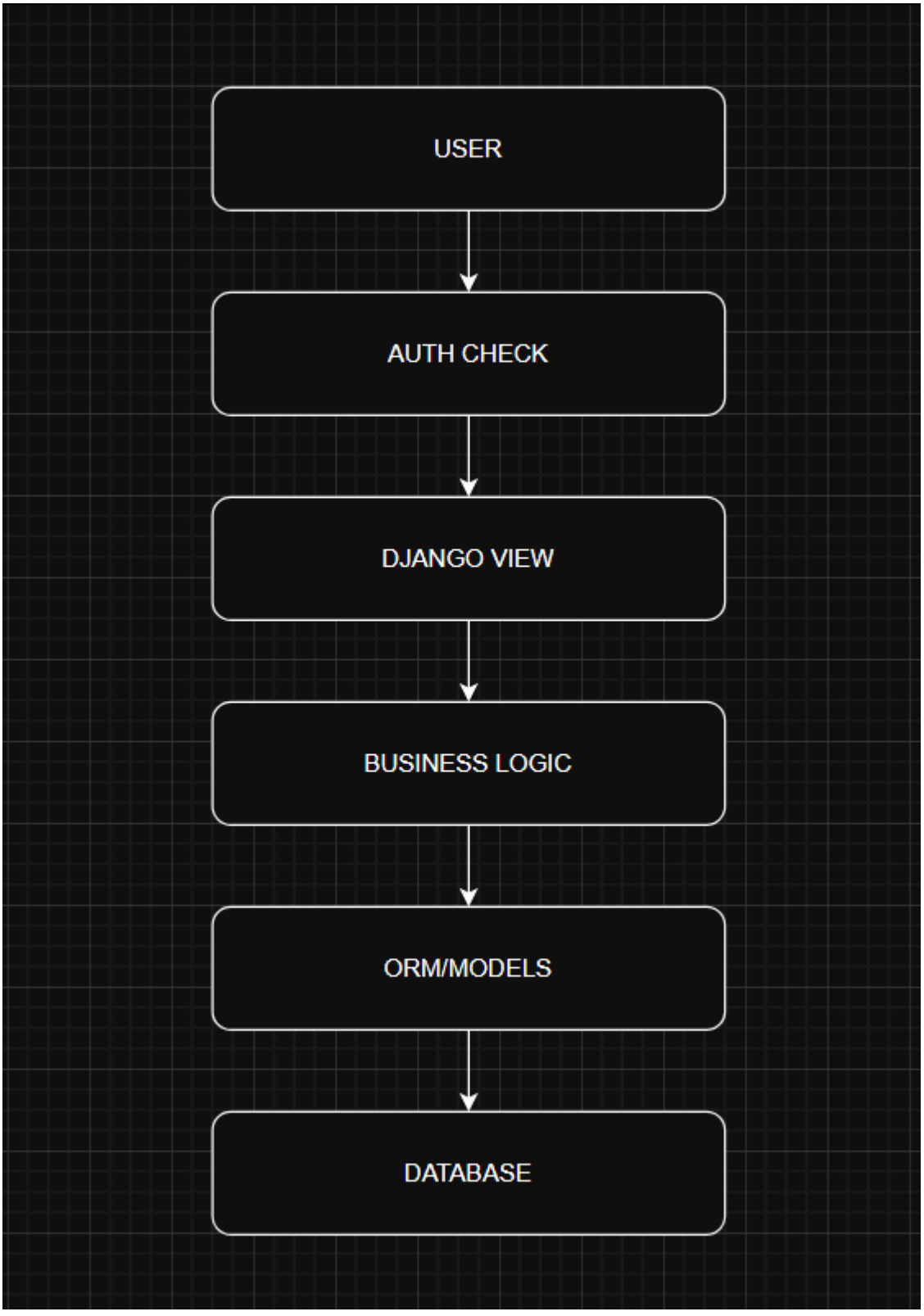
3. Conceptual DB Sketch (Entity Diagram)



4. Data Access Boundaries

Action	Item	Transaction	User
Create Item	W	–	–
Update Item	W	–	–
Delete Item	W	–	–
View Items	R	–	–
IN Transaction	W	W	R
OUT Transaction	W	W	R
View Transactions	R	R	R
Low Stock View	R	–	–
Analytics	R	R	–

5. System Flow



6. Transaction Flow Logic (Out Flow)

```
Start
↓
User submits request
↓
Is user authorized?
↓
Is quantity > 0?
↓
Is stock sufficient?
↓
Update Item.quantity
↓
Create Transaction
↓
End
```

Tech stack

> Python + Django + PostgreSQL + Django Templates + GitHub + Render

Core Backend

Language

- **Python 3.x**

  Reason: Mature ecosystem, readability, first-class Django support.

Framework

- **Django**

  Reason:

  Built-in ORM (perfect for relational data)

  Authentication & permissions out of the box

  Admin panel for rapid CRUD

  Secure defaults (CSRF, SQL injection protection)

Database

Development

- **SQLite**

  Reason:

  Zero configuration

  Ideal for local development and learning

Production

- **PostgreSQL**

  Reason:

  Strong relational integrity

  ACID compliance

  Excellent Django integration

  Industry standard for backend systems

Frontend (Server-Rendered)

Templating

Django Templates (DTL)

Reason:

Tight backend integration

No frontend framework overhead

Suitable for admin-style dashboards

Styling

- **HTML5 + CSS3**
- JS

Authentication & Authorization

Django Auth System

Reason:

User management

Password hashing

Groups & permissions

Role-based access control

Data Handling & Logic

- **Django ORM**
- **Model-level validations**
- **Database transactions (`atomic`)**

Reason:

- Prevents inconsistent state
- Keeps business logic server-side

Version Control

- **Git**
- **GitHub**

Reason:

Industry expectation

Enables collaboration and CI later

Deployment & Infrastructure

Hosting

- **Render** or **Railway**

  Reason:

  Simple Django deployment

  Managed PostgreSQL

  Minimal DevOps overhead

Web Server

- **Gunicorn**

  Reason:

  Production-grade WSGI server for Django

Static Files

- **Whitenoise**

  Reason:

  Serve static files without extra services

Environment & Configuration

Python Virtual Environment

Environment Variables (`.env`)

Django Settings Split (dev / prod)

Reason:

Security

Clean configuration management